

BACKEND
DEVELOPER

Portfolio

임성제 포트폴리오



I About me

트래픽 처리와 시스템 확장성을 고민하며 백엔드와
클라우드 인프라까지 함께 만드는 개발자
임성제 입니다.

트래픽 증가 상황에서도 안정적으로 동작하는 서비스를 만드는 데 관심이 있습니다.

REDIS SORTED SET 기반 대기열, KAFKA 기반 순서 보장, CQRS, REDIS CACHE, KUBERNETES 환경 구축 등의 경험을 통해 단순 기능 구현을 넘어 성능과 확장성을 고려한 백엔드 시스템을 설계하고 있습니다.

백엔드 개발부터 클라우드 인프라 구축까지 서비스의 전체 생명주기를 경험하며 개발부터 배포까지 수행할 수 있는 개발자입니다.

I About Me

임성제

2000.04.16

명지대학교 융합소프트웨어학부 응용소프트웨어 전공

I Channel



<https://github.com/seongje00416>

<https://github.com/dalgurum>



<https://cloud-wiki.tistory.com/>



<https://central.sonatype.com/namespace/io.github.dalgurum>

I Skills

Backend | Spring Boot Spring WebFlux Redis Kafka gRPC

Database | MySQL PostgreSQL MongoDB DynamoDB

Infra | AWS Kubernetes ArgoCD Github Actions Terraform

DevOps | Github Jira Confluence Notion Slack

I Work

(재)국제인재능력개발원

2026.06 ~ 2026.07

네이버 클라우드

2024.04 ~ 2024.08

(주) 스프레틱스

2020.07 ~ 2020.10

I Education

[KDT] AI 클라우드 네이티브 개발자 양성 과정

메가존 클라우드

2025.09 ~ 2026.03

Project

팀 프로젝트
2026.01 – 2026.03



★ Now&Go
서버 개발자

여행 올인원 플랫폼

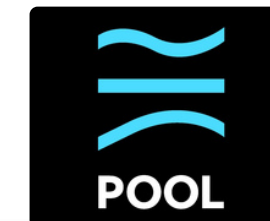
팀 프로젝트
2025.03 – 2026.01



★ Fooding
백엔드 개발자

요식업 올인원 플랫폼

팀 프로젝트
2025.03 – 진행 중



Pool
서버 개발자

숯폼 기반
리퀘스트 매칭 플랫폼

팀 프로젝트
2025.03 – 2025.06



학식로그
PM / 클라우드 엔지니어

학생들을 위한 학식 정보
전달 서비스



여행 계획부터 티켓 발급까지 가능한 트래블 올인원 서비스

역할 : 백엔드 개발 및 클라우드 인프라 구축

작업 기간 : 2026.01 ~ 2026.03 (약 2개월)

인원 : 3명

링크

<https://github.com/seongje00416/now-n-go-be.git>

기술 스택

Spring Boot Redis AWS Kubernetes PostgreSQL Github ArgoCD

성과

Redis SortedSet 기반 대기열 구현을 통해 동시 사용자 3,000명의 환경에서 에러율을 80%에서 0%로 개선

I 좌석 선점 대기열

문제

- 동일 상품을 동시에 구매시 초과 판매 가능
- DB Connection Pool 고갈 우려

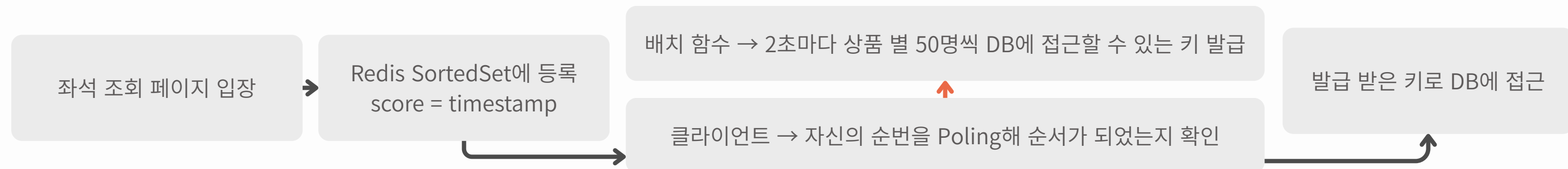
해결 방안 탐색

방법 1) DB 대기열을 사용 → DB 부하 증가

방법 2) Kafka를 사용 → 기능 대비 과도한 복잡도

방법 3) Redis SortedSet을 사용 → 빠른 조회 + 순번 관리 가능 / 추가적인 인프라 구축 필요

적용



결과

k6 시나리오 : 동시 사용자 3,000명 접속 시도 5회 측정

(기존) DB Connection Pool 소진 에러 발생, 에러율 80%(5회 중 4회 발생) → (개선) 에러 없이 정상 작동 (5회 중 0회 발생)

첨부

시연 영상 주소: https://youtu.be/DuvTqRtna-k?si=fFBEw_vv-KNdWbi5

I CQRS 패턴 적용

- 문제 : 데이터 서비스에 트래픽이 몰려 성능이 저하
- 해결 방안 탐색 : 빠른 응답이 목표인 조회 기능 + 데이터 정합성이 최우선인 쓰기 기능 → 두 기능을 분리할 수 있음
- 적용 : 데이터 서비스를 Read 서비스와 Write 서비스로 분리
- 결과 : Read 서비스에 대해 독립적 Scale-Out이 가능해 선택적 확장 가능한 구조 확보

I 분산 보상 트랜잭션 구현

- 문제 : 두 개 이상의 테이블에서 쓰기 작업을 수행하다 중간에 실패했을 때 고아 데이터가 생김
- 해결 방안 탐색 : MSA 구조상 DB 트랜잭션으로 묶을 수 없음 → 직접 보상 트랜잭션 구현 필요
- 적용 : 각 단계에 대한 성공 결과를 해당 데이터의 ID로 저장해두었다가 실패시 try-catch문을 통해 성공한 단계의 역순으로 데이터를 지우는 작업 수행
- 결과 :
 - sku-flight 데이터 생성 과정에서의 변화 - product 데이터 생성 → skus 데이터 생성 → sku-flight 데이터 생성
 - 기존: sku-flight 데이터 생성 중 실패시 product 데이터와 skus 데이터가 남아 2개의 고아 데이터 존재
 - 개선: sku-flight 데이터 생성 중 실패해도 보상 로직으로 통해 앞선 데이터를 삭제해 0개의 고아 데이터 존재

I Fooding

외식업 운영의 모든 것을 하나로 연결하는 올인원 플랫폼

역할 : 백엔드 개발

작업 기간 : 2025.03 ~ 2026.01 (약 10개월)

인원 : 10명

링크

Fooding: <https://github.com/FORCU27/fooding.git>

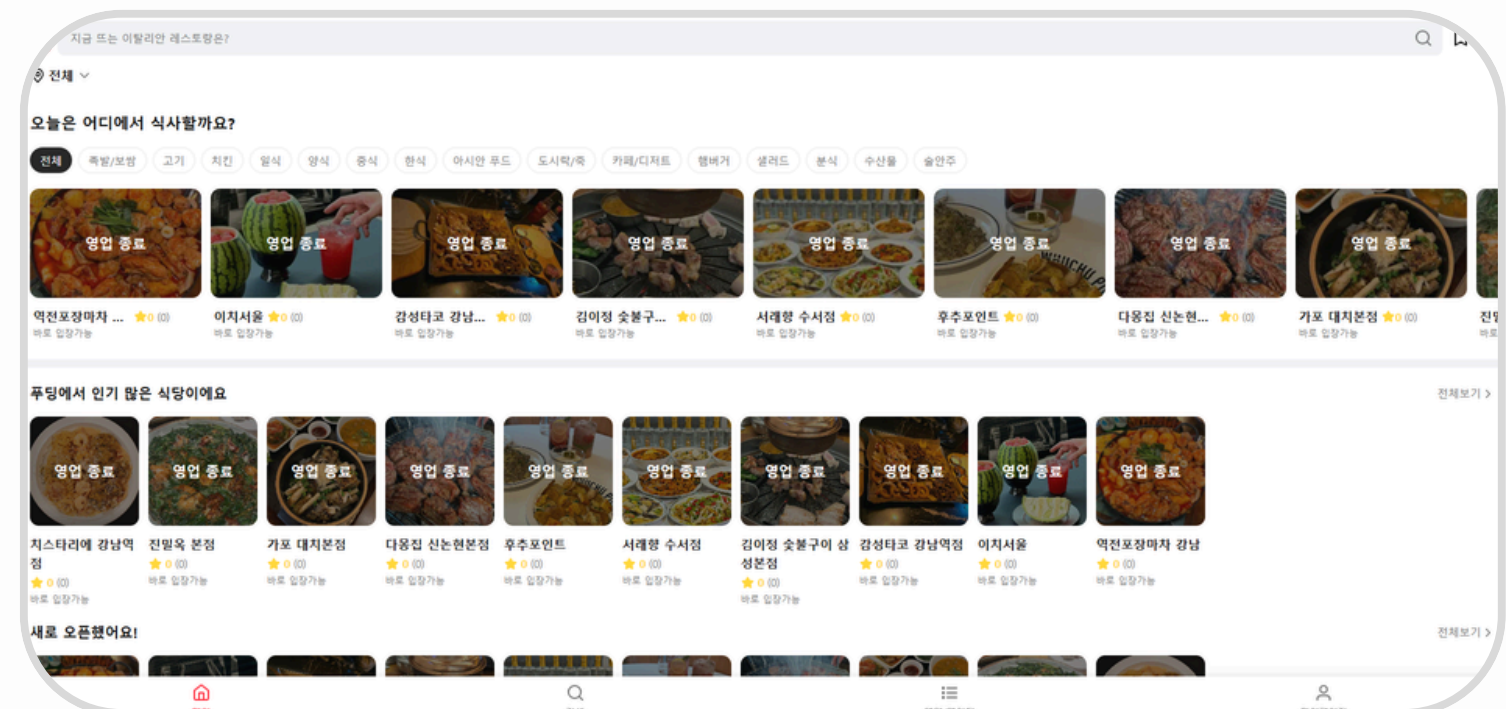
BackEnd: <https://github.com/seongje00416/fooding-backend.git>

기술 스택

Spring Boot Redis WebFlux Kafka MySQL MongoDB Github Jira

성과

Redis Cache 도입으로 평균 응답 시간 43% 감소



I Redis 캐시 적용에 따른 성능 개선

문제

메인 화면에서 호출하는 API 4개가 모두 DB 풀스캔 수행 → 트래픽이 몰릴 경우 DB 과부하 발생 가능

해결 방안 탐색

방법 1) Caffeine 로컬 캐시 사용 → 서버 재시작시 캐시 데이터 소멸 + 다중 인스턴스 환경에서 사용 불가

방법 2) DB 자체 캐싱 → 설정 복잡도가 높음

방법 3) Redis 분산 캐시 사용 → 다중 인스턴스 환경 사용 가능 + 설정 쉬움 / 별도 인프라 구축 필요

적용

개선 1) 메인 화면을 위한 API 엔드포인트 분리 → 기존에는 하나의 공용 가게 조회 API에 파라미터 값만 조절해서 조회하는 방식 사용

개선 2) 분리한 엔드포인트에 Redis 캐시 적용

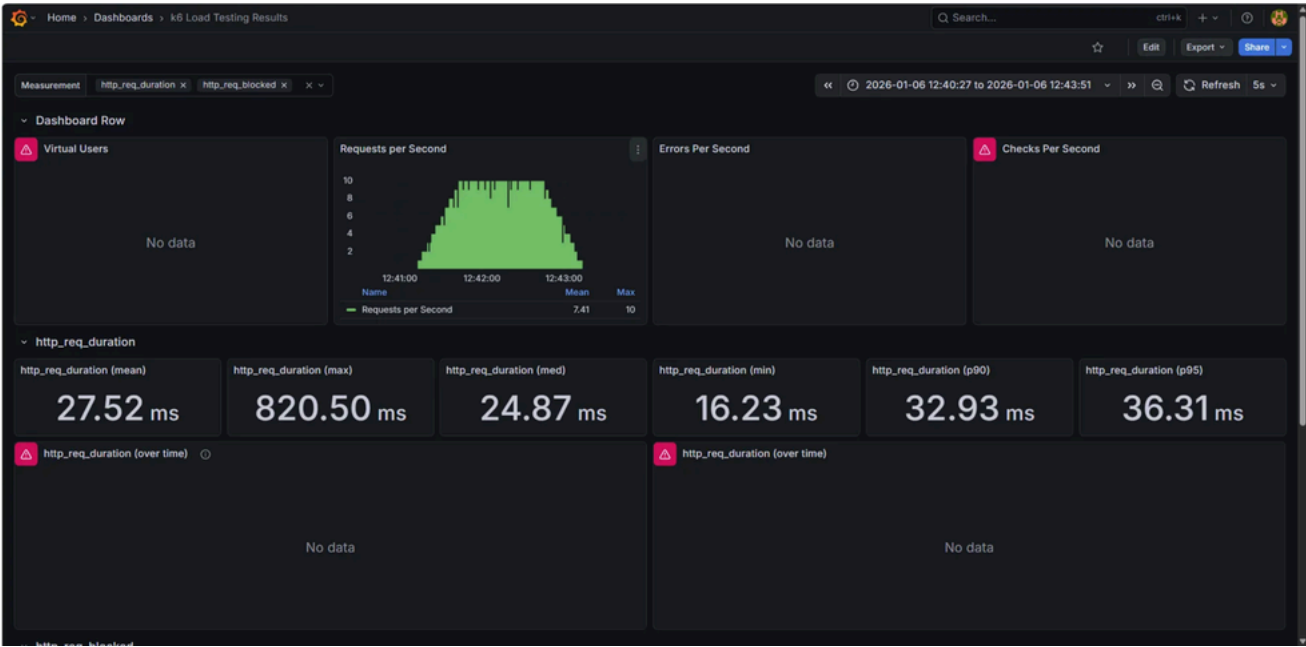
개선 3) @Cacheable(sync = true) 옵션 적용 → Cache Stampede 방지

전체 가게 데이터	가게 목록에 대한 Create, Update, Delete 작업이 일어난 경우	TTL(15분)이 지난 경우
인기 가게 목록	조회수를 기준으로 한 상위 인기 가게 순위에 변동이 생긴 경우	TTL(15분)이 지난 경우
신규 입점 가게 목록	새로운 가게 등록 작업이 일어난 경우(관리자의 승인이 된 경우)	TTL(15분)이 지난 경우
바로 입장 가능한 가게 목록	대기열이 새로 생긴 가게가 있는 경우	TTL(15분)이 지난 경우

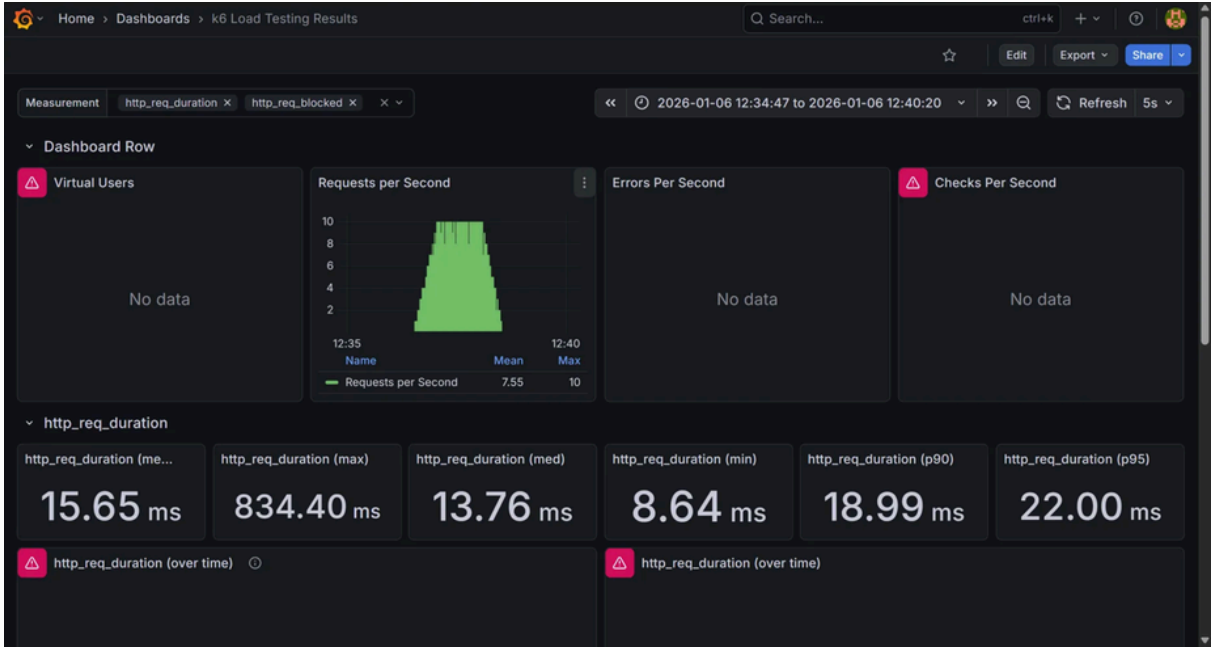
캐시 데이터 초기화 기준

Redis 캐시 적용에 따른 성능 개선

결과



캐시 미적용



캐시 적용

	평균	중간값	P90	P95
캐시 미적용	27.52ms	24.87ms	32.93ms	36.31ms
캐시 적용	15.65ms	13.76ms	18.99ms	22.00ms
개선율	43% 감소	45% 감소	42% 감소	39% 감소

최대 응답 시간은 캐시 만료 후 최소 DB 접근 시 발생하는 캐시 미스의 영향으로 소폭 증가했으나, 실제 사용자 경험에 영향을 주는 평균 및 P95와 같은 값들은 모두 약 40% 이상 개선

Ⅰ 사용자 웨이팅 기능 동시성 제어

문제

- 동일한 상품에 동시에 예약이 발생하거나 사용자가 많아지면 Race Condition 문제 발생 가능
- 현재 대기열 기능으로는 웨이팅 번호가 중복되거나 순서가 뒤섞이는 데이터 정합성 문제 발생 가능

해결 방안 탐색

방법 1) synchronized 사용 → 다중 서버 환경에서 작동 X

방법 2) DB의 Pessimistic Lock 사용 → Lock 경합이 DB에 집중되어 성능 저하 유발

방법 3) Redis를 사용한 분산락 적용 → 다중 서버 환경에서도 동작 + 빠른 응답 가능 / 별도 인프라 구축 필요

적용

1차 고도화 : Redisson 분산락을 통한 정합성 확보

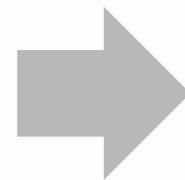
- 한계 → WAS와 DB가 장시간 점유되어 시스템 장애 유발 가능 + 완벽한 순서 보장 불가

2차 고도화 : Kafka 대기열을 통한 순서 보장 및 부하 제어

- store의 ID 값을 파티션 키로 사용 → 동일한 매장에 대한 예약 요청에 엄격한 FIFO 순서 보장
- Consumer가 처리 가능한 속도로 메시지를 소비 → DB와 WAS 점유로 장애를 일으킬 수 있었던 문제 해결

결과

k6를 통한 테스트
동시 요청 개수: 100개
횟수: 5회



기존

중복 웨이팅 번호 수 : 2개

2차 고도화 완료

중복 웨이팅 번호 수 : 0개

I 실시간 알림 서비스 모듈 개발

문제

웨이팅 결과를 POS 화면에 실시간으로 띄우는 기능 개발이 필요 → 현재 서비스는 Spring MVC로 작동해 실시간 정보 교환에 적합하지 않음

해결 방안 탐색

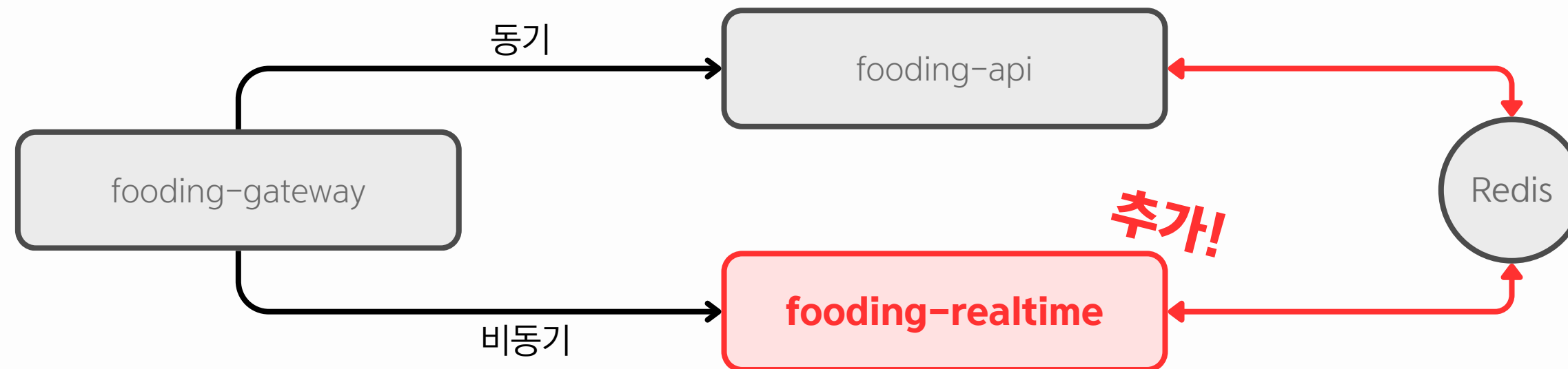
방법 1) 주기적 Polling → 불필요한 요청이 많아짐

방법 2) WebSocket → 양방향 통신이 필요 없는 알림 서비스 특성상 오버스펙

방법 3) SSE 활용 → Spring MVC 환경에선 쓰레드 고갈 우려

방법 4) WebFlux 비동기 모듈 개발 → SSE의 장점을 가져가며 Spring MVC 환경의 한계 극복 가능

적용



결과

- 기존 Spring MVC 환경을 유지하기 때문에 기존 서비스 수정 없이 실시간 알림 기능 추가
- 실시간 알림이 필요한 기능 개발시 SSE를 적극 활용 가능

I Pool



Short-Form 기반의 Request 매칭 플랫폼

역할 : 백엔드 개발 및 클라우드 인프라 구축

작업 기간 : 2025.03 ~ 진행 중

인원 : 9명

기술 스택

Spring Boot Lambda AWS MySQL gRPC Claude API Github Notion

성과

AI 기반 장애 분석 시스템 구축으로 트러블슈팅 시간을 2시간 → 20분으로 단축

I AI 모니터링 에이전트 구현

문제

- 실시간 관제 불가 → 문제 인지까지 시간이 오래 걸림
- 트러블슈팅이 이루어지는데 시간이 오래 걸림 → 문제 인지 - 원인 분석 - 해결 방안 도출 및 적용 - 배포

해결 방안 탐색

문제 인지까지 걸리는 시간 단축 → 알림 서비스 활용
원인 분석 및 해결 방안 도출 → AI를 활용

적용

- 서비스 FilterChain에 Claude API를 연동 → 에러 발생시 Claude AI를 통해 원인 분석 및 해결 방안 제안 수집
- Discord 웹훅 사용 → 에러 로그, AI 분석 결과를 Discord를 통해 알림으로 수신

결과

- QA 기간 내 발생한 에러 17건 기준 트러블슈팅 완료까지 걸리는 시간이 평균 2시간 → 20분으로 단축

```
# -서버-에러
org.springframework.dao.support.PersistenceExceptionTranslationInterceptor.invoke(PersistenceExceptionTranslationInterceptor.java:160)
AI 원인 분석 및 해결 방법
# 예외 원인 및 해결 방법

## 1. 가능한 원인
[redacted] 컬럼의 데이터 길이 초과
• 데이터베이스의 [redacted] 컬럼 크기가 저장하려는 값 [redacted] 보다 작음
• 예: VARCHAR(3) 크기 컬럼에 VARCHAR(6) 이상의 값을 저장하려고 함

원인 분석:
요청값: [redacted] (6자)
DB 컬럼: VARCHAR(5) 이하로 설정됨
→ Data truncated 에러 발생

---

## 2. 해결 방법

### 방법 1: 데이터베이스 스키마 수정 (권장)
[redacted]

### 방법 2: Entity 검증
@Entity
@Table(name = [redacted])
public class [redacted] {
    @Column(name = "[redacted]", length = 50) //
    적절한 길이 설정
    private String [redacted];
}

### 방법 3: Enum 사용 (권장)
@Entity
@Table(name = "[redacted]")
public class [redacted] {
    @Enumerated(EnumType.STRING)
    @Column(name = "target_type", length = 50)
}
```

I gRPC를 사용한 채팅 서비스 구현

문제

- 앱 클라이언트(Swift, Kotlin)에서 사용 가능한 텍스트 채팅 서비스 구현

해결 방안 탐색

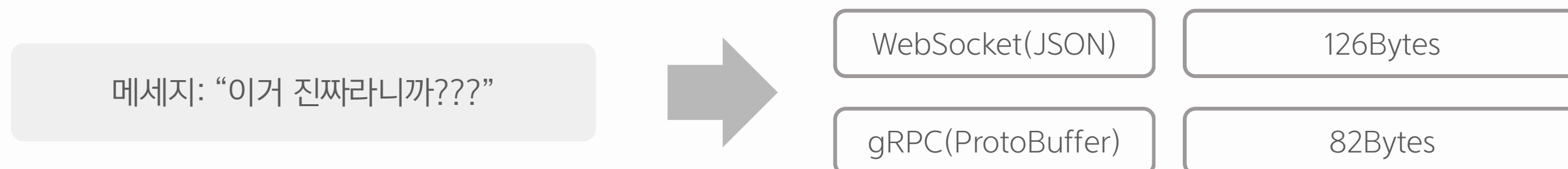


적용

- gRPC를 사용한 텍스트 채팅 서비스 구현
 - proto를 통해 iOS와 Android에서 동일한 통신 규약 적용 → proto를 통한 코드 자동 생성 + 컴파일 단계에서의 타입 안정성 확인

결과

- 페이로드의 크기가 WebSocket에 비해 줄어들어 효율적인 채팅 서비스 구현



ㅣ 학식로그

학생들의 학식 사용 장려 및 발전을 위한 서비스

역할 : PM 및 서비스 인프라 구축

작업 기간 : 2025.03 ~ 2025.06

인원 : 30명

기술 스택

AWS Kubernetes Github Github Actions ArgoCD Slack Google Sheet

성과

30명 규모 팀의 CI/CD 완전 자동화로 인한 배포 시간 단축

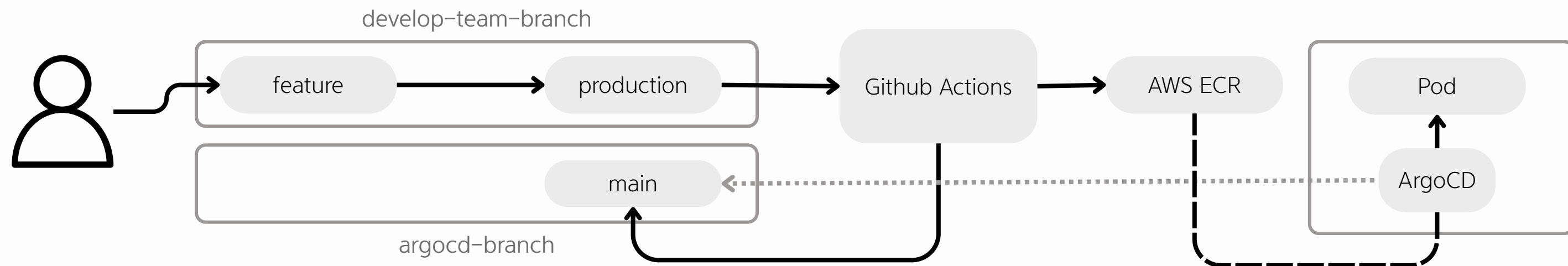


I Kubernetes 환경 구축

- 문제 : 팀원 or 팀별 독립적인 개발 환경 필요
- 해결 방안 탐색
 - 모듈러 모놀리식 구조 - 도메인 별 독립적인 개발 + 개발 환경에 대한 낮은 러닝 커브 / 공통 모듈 or 설정 파일 수정시 다른 서비스에도 영향을 줄 가능성 높음
 - 마이크로 서비스 아키텍처 - 도메인 별 독립적인 개발 + 완전히 독립적인 개발 환경 / 새로운 구조에 대한 러닝 커브 높음
- 적용
 - 마이크로 서비스 아키텍처 채택 → 독립적인 개발 환경의 중요성 > 러닝 커브의 존재
 - Kubernetes를 통한 서비스 배포 → 러닝 커브 최소화를 위해 AWS EKS 활용

I 모든 CI/CD 파이프라인 자동화

- 문제:
 - 개발팀 - Kubernetes 및 MSA 구조에 대한 높은 러닝 커브 존재
 - 인프라팀 - 서비스 배포에 많은 시간 소비
- 해결 방안 탐색 : CI/CD 과정 자동화 → 개발팀의 러닝 커브 문제와 인프라팀의 시간 소비 문제를 모두 해결할 수 있는 방향
- 적용



I Contact me

더 자세한 개인 정보는 Github 및
이력서를 참고해주세요. 감사합니다.



Github

<https://github.com/seongje00416>



이메일

seongje00416@gmail.com

THE END

Thank you

감사합니다